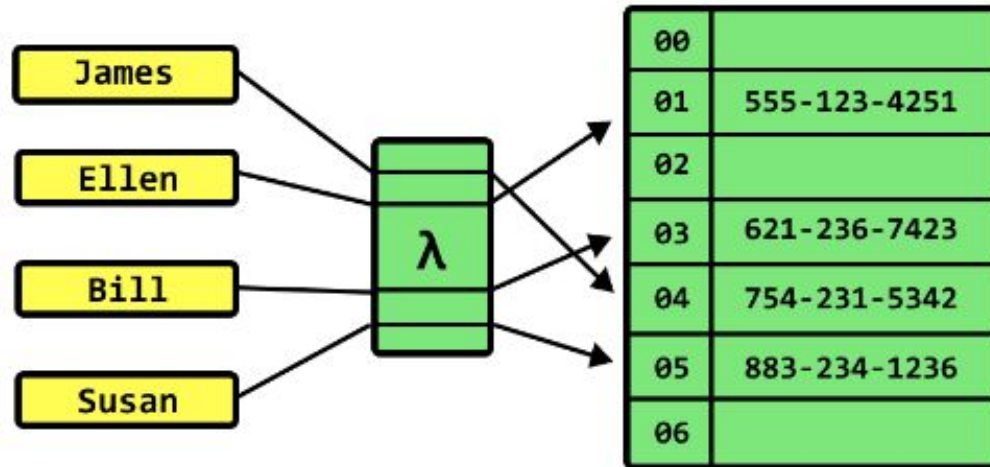




ED1 07: Hash Tables

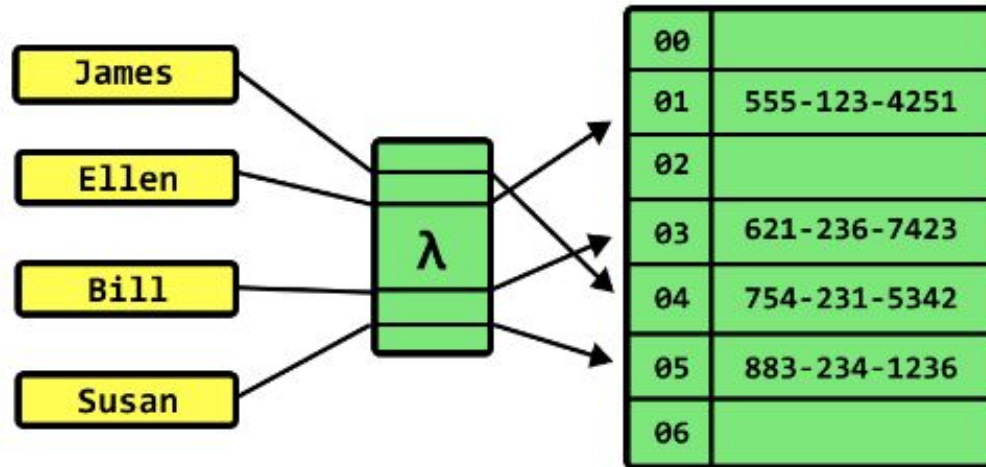
Fabio Irigon Pereira

Hash Tables

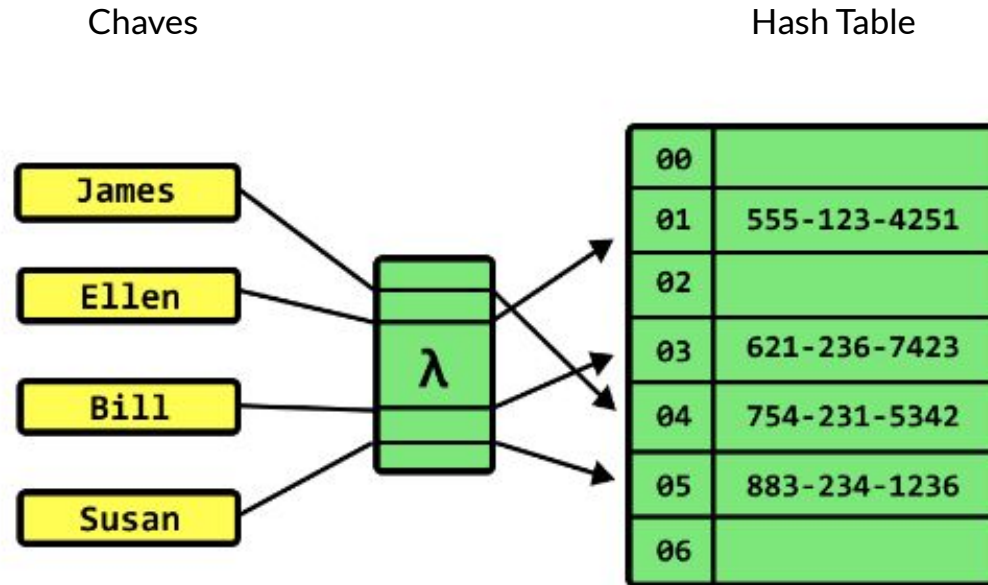


Hash Tables

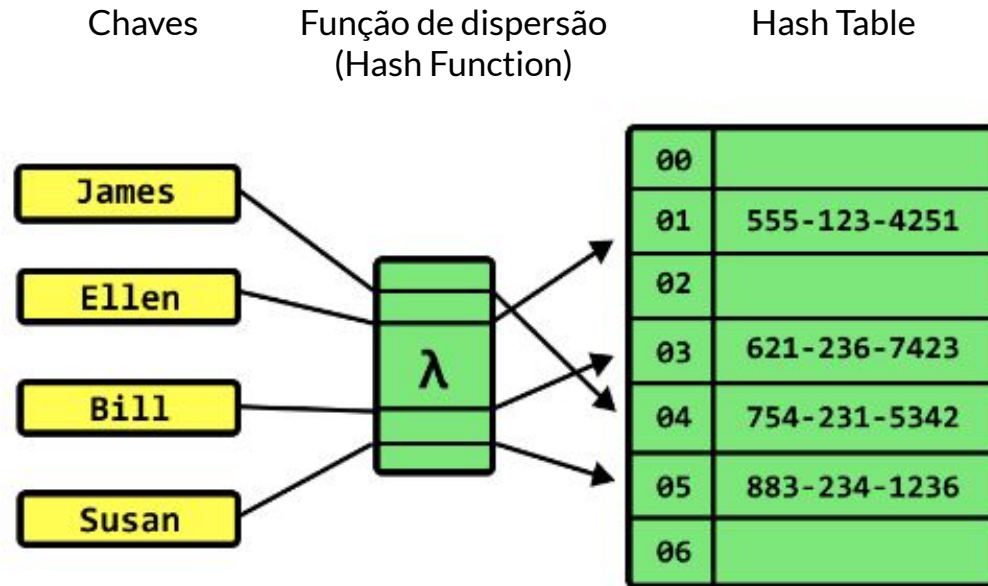
Chaves



Hash Tables



Hash Tables



Conteúdo



- Conceito de **funções de dispersão** (hash functions)
- Estrutura e funcionamento de **tabelas hash**
- Técnicas de resolução de **colisões**
- Implementar uma tabela hash simples

Motivação



Imagine que você precise procurar um contato em uma lista telefônica ou recuperar um CPF associado a uma chave.

Estrutura	Tempo Médio
Lista/Array	$O(n)$
Lista Ordenada	$O(\log n)$
Árvore de busca	$O(\log n)$
Tabela Hash	$O(1)$

Exemplos de uso



- Dicionários de Idioma (Word Processors)
- Compiladores e Interpretadores
- Sistemas de Cache (Web Browsers, Servidores)
- Dicionários em python (internamente)

Estrutura tabela hash



- Um array.
- Uma função hash.
- Estratégia de tratamento de colisões.

Função Hash



Transforma uma chave (string ou int) em um índice de array (int).

Objetivos:

- **Rapidez:** deve ser computacionalmente leve
- **Distribuição uniforme:** minimizar colisões
- **Determinismo:** mesma entrada, mesma saída sempre
- Evitar agrupamentos de chaves parecidas

Exemplos:



Um exemplo simples é usar o resto da divisão inteira (%).
<https://www.cs.usfca.edu/~galles/visualization/OpenHash.html>

```
int hash(int chave, int tamanho) {  
    return chave % tamanho;  
}
```

```
int hash(char *str, int tamanho) {  
    int soma = 0;  
    for (int i = 0; str[i] != '\0'; i++)  
        soma += str[i];  
    return soma % tamanho;  
}
```

Tabela exemplo



```
[0] -> NULL
[1] -> NULL
[2] -> "Alice"
[3] -> NULL
[4] -> "Bob"
[5] -> NULL
[6] -> NULL
[7] -> NULL
[8] -> NULL
[9] -> NULL
```

```
int hash(char *str) {
    return djb2(str) % 10;
}
```

Exemplo real:



```
unsigned long djb2(char *str) {  
    unsigned long hash = 5381;  
  
    int c;  
  
    while ((c = *str++))  
        hash = ((hash << 5) + hash) + c; // hash * 33 + c  
  
    return hash;  
}  
// usada na glib
```

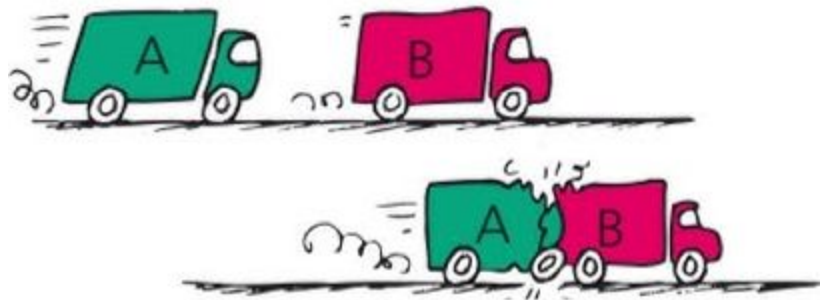
Colisões

Duas chaves geram o mesmo índice.

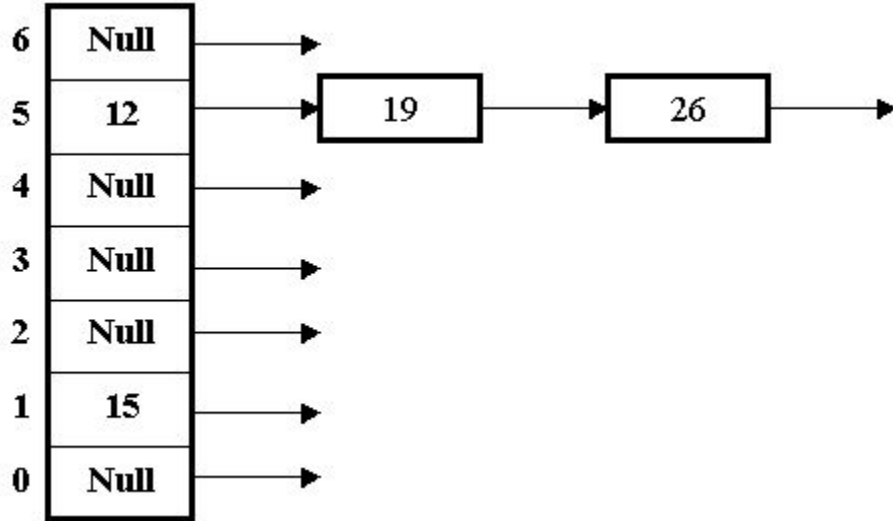
```
hash("Alice") % 10 == 2
```

```
hash("Carol") % 10 == 2
```

Duas opções: **encadeamento** ou **endereçamento aberto**.



Encadeamento



```
typedef struct No {  
    char *chave;  
    char *valor;  
    struct No *prox;  
} No;
```

```
typedef struct {  
    No tabela[MAX];  
} HashTable;
```

Endereçamento aberto



Em caso de colisão busca-se a próxima posição livre.

Estratégias:

Linear Probing: $\text{indice} + 1, +2, \dots$

Quadratic Probing: $\text{indice} + 1^2, +2^2, \dots$

Double Hashing: usa uma segunda função hash para pular

Fator de carga (Load Factor)



É a razão entre o número de elementos e o tamanho da tabela:

$$\alpha = \frac{n}{m}$$

- Ideal: $\alpha < 0.75$ (no caso de endereçamento aberto)
- Um α alto reduz muito a performance.